

APPENDIX A LABELED TRANSITION SYSTEM

We define the TLS transition system as $M_{\text{TLS}} = (S, A, \rightarrow, s_0)$, where S is the set of well-formed TLS object configurations, A is the set of rewrite-rule applications, $\rightarrow \subseteq S \times A \times S$ is the transition relation, and $s_0 \in S$ is the initial configuration. Each state contains the client and server endpoint objects and the two directional link objects.

An action specifies the context of a rewrite-rule application:

$$a = (e, \ell, m, \text{objs}) \in A.$$

Here $e \in \{\text{send}, \text{receive}, \text{error}\}$ is the event kind. The component ℓ is the Maude rewrite-rule label, m is the TLS message type handled by the rule, and objs records the Maude object terms matched by the rule's left-hand side.

The transition relation $s \xrightarrow{a} s'$ holds when a conditional Maude rewrite rule with rule label ℓ , event kind e , and handled message type m matches the objects objs in configuration s , its condition is satisfied, and applying the rule rewrites s to s' . The action a is not an additional semantic step; it is the recorded context of the rewrite step.

For example, applying the `sendClientHello` rule to the client object obj_c and the client-to-server link object obj_{c2s} is associated with

$$(\text{send}, \text{sendClientHello}, \text{ClientHello}, \{\text{obj}_c, \text{obj}_{c2s}\}).$$

The successor state contains the updated client object and the constructed `ClientHello` appended to the client-to-server queue. Applying the corresponding `receiveClientHello` rule to obj_{c2s} and the server object obj_s is associated with

$$(\text{receive}, \text{receiveClientHello}, \text{ClientHello}, \{\text{obj}_{c2s}, \text{obj}_s\}).$$

The successor state contains the consumed queue entry and the updated server object.

A finite TLS trace is an alternating sequence of states and actions:

$$\tau = s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} \dots \xrightarrow{a_{k-1}} s_k.$$

APPENDIX B REUSABLE BEHAVIOR-DEVIATION PATTERNS

RFC-conformance testing spans many TLS message types, extensions, fields, and protocol contexts, but the framework does not try to enumerate every malformed input manually. Instead, for specification-violation testing, it provides four parameterized tester-side behavior-deviation patterns that capture recurring off-specification message-production behaviors within the modeled TLS handshake scope. These patterns form a reusable generation library, not an exhaustive taxonomy of TLS bugs: each pattern specifies one class of malformed tester behavior, and the framework instantiates that class across concrete TLS contexts by enumerating typed parameter domains, using built-in domains for framework-provided types and user-provided domains for user-defined types.

- **P1:** `addUnexpectedExtension` inserts an extension component that is disallowed in the current message or protocol context.

- **P2:** `removeNecessaryExtension` omits an extension required by the current message or protocol context.
- **P3:** `changeMessageFieldValues` replaces a TLS message-field value while preserving the structure.
- **P4:** `skipExpectedMessage` omits a message expected in the current modeled handshake path.

For example, P1 captures extension-validity violations such as duplicate extensions, forbidden extensions in a message, and unsolicited extension responses. It also covers extensions that are valid in one handshake message but invalid in another. The pattern selects send rules for a chosen TLS message type and inserts an extension field-value component into the constructed message. The expression `#getType($extField)` returns the value type associated with the selected extension field, so each instantiated `$extValue` is type-consistent with `$extField`. Example parameter instances include adding a `signature_algorithms` extension to a `ServerHello` or adding a `supported_groups` extension to a `CertificateRequest`.

BehaviorId	: <code>addUnexpectedExtension</code>
Parameters	: <code>\$extField : ExtensionField,</code> <code>\$extValue : #getType(\$extField), \$msgType : TLSType</code>
Conditions	: <code>event = send and ruleMsgType = \$msgType</code>
Modification	: <code>add(\$extField, \$extValue)</code>

P2 captures missing-extension violations in contexts where the RFC-compliant peer expects mandatory extension information. These cases differ from P1 because the malformed message is invalid by omission rather than by the presence of an unexpected component. The pattern selects send rules for a chosen TLS message type and removes the selected extension field from the constructed message. Example parameter instances include removing `key_share` from a `ClientHello` in a context where that extension is required or removing `signature_algorithms` from a `CertificateRequest`.

BehaviorId	: <code>removeNecessaryExtension</code>
Parameters	: <code>\$extField : ExtensionField,</code> <code>\$msgType : TLSType</code>
Conditions	: <code>event = send and ruleMsgType = \$msgType</code>
Modification	: <code>remove(\$extField)</code>

P3 captures value-validity violations in modeled TLS message fields while preserving the corresponding message structure. The pattern selects send rules for a chosen TLS message type and replaces a selected message field with a type-consistent replacement value. This allows one specification to cover deviations in negotiated values, authentication-related fields, and other modeled TLS fields. The deviation remains at the structured-message level: it changes the field value represented in the Maude message term, not the concrete byte encoding. Example parameter instances include setting `legacy_version` in a `ClientHello` to `TLS11` or changing the `handshakeType` of a `CertificateRequest` to `ServerHello`.

BehaviorId	: <code>changeMessageFieldValues</code>
Parameters	: <code>\$msgField : MessageField,</code> <code>\$msgValue : #getType(\$msgField), \$msgType : TLSType</code>
Conditions	: <code>event = send and ruleMsgType = \$msgType</code>
Modification	: <code>setM(\$msgField, \$msgValue)</code>

P4 captures missing-message violations in contexts where the current modeled handshake path requires a particular message next. Once a TLS handshake path is determined, the peer has an expected message order; if the expected message does not appear, the receiver observes an RFC-deviating message flow. Unlike P1–P3, which modify the contents or structure of a message, P4 selects send rules for a chosen TLS message type and omits the corresponding message.

More complex ordering scenarios, such as replaying a previous message, inserting an additional message, or sending a later message in place of the omitted one, can be constructed by combining multiple deviations and scenario properties. They are therefore not represented as a single generic reordering primitive. An example parameter instance omits a server Certificate message.

BehaviorId	: skipExpectedMessage
Parameters	: \$msgType : TLSMsgType
Conditions	: event = send and ruleMsgType = \$msgType
Modification	: skip()

APPENDIX C

RFC 5246 AND RFC 8446 REQUIREMENT COVERAGE

We model RFC requirements at the level of individual MUST and MUST NOT obligations because the Maude model is used as an executable oracle for RFC-conformance test generation, not merely as a high-level protocol sketch. A generated test needs a concrete reason why a message is valid or invalid in a particular handshake context, and the framework must know what a compliant endpoint should do next. Encoding RFC MUST and MUST NOT requirements as explicit send-side construction constraints, receive-side checks, error transitions, or state updates lets the framework derive targeted malformed variants, predict the expected rejection behavior, and relate an implementation’s unexpected acceptance back to the violated RFC requirement.

The coverage tables below make this oracle boundary inspectable at the requirement-category level. In each table, rows organize RFC 5246 or RFC 8446 MUST and MUST NOT requirements by protocol function. The **Reqs.** column reports the number of requirements in the corresponding requirement category, the **Model** column reports how many of those requirements are explicitly encoded in the Maude model, and the **Cov.** column reports the coverage ratio. A requirement contributes to the **Model** column only when the executable Maude model encodes that requirement as logic for constructing a TLS message, validating a received message, updating handshake state, or rejecting invalid input.

Table II shows that the model covers 86 of 129 included RFC 5246 requirements. The covered requirements are concentrated in the TLS 1.2 categories that determine endpoint-visible handshake behavior: message ordering, extension validity, negotiation consistency, authentication validity, cryptographic context, and session/resumption handling. These are the requirements that the model must encode to construct valid handshake traces, generate malformed variants, and decide whether a peer should accept or reject a message. Coverage is

lowest in the *Others* category because many requirements there specify fragmentation, compression, MAC and encryption processing, cipher-suite definitions, deployment compatibility constraints, or other details outside the handshake-level oracle used by our generated tests.

Table I shows that the model covers 229 of 327 included RFC 8446 requirements. The covered requirements are concentrated in the TLS 1.3 categories that determine endpoint-visible handshake behavior: message ordering, extension validity, negotiation consistency, authentication validity, cryptographic context, and session/resumption/post-handshake behavior. These are the requirements that the model must encode to construct valid handshake traces, generate malformed variants, and decide whether a peer should accept or reject a message. Coverage is lowest in the *Others* category because many requirements there specify byte-level encoding, cryptographic computations, anti-replay or deployment obligations, or other details outside the handshake-level oracle used by our generated tests.

APPENDIX D

SEMANTICS OF STATE AND ACTION PROPOSITIONS

Scenario properties constrain finite traces of the labeled transition system in Section A. They use two kinds of predicates. State propositions are evaluated over TLS object configurations, and action propositions are evaluated over the recorded context of rewrite-rule applications. This separation lets a scenario refer both to protocol states, such as an endpoint reaching *error* or *done*, and to transition events, such as applying a particular deviated rule or taking an error transition.

Let $s \in S$ be a TLS object configuration. Since configurations are multisets of Maude objects, we use $\{\langle o : C \mid x : v, \dots \rangle, \dots\}$ to denote a configuration that contains an object with identifier o , class C , and attribute x with value v ; the omitted attributes and the other objects in the configuration are irrelevant to the proposition. The satisfaction relation for state propositions is:

$$\begin{aligned}
s \models o.x = v &\iff s = \{\langle o : C \mid x : v, \dots \rangle, \dots\}, \\
s \models \text{not } \delta &\iff \neg(s \models \delta), \\
s \models \delta_1 \text{ and } \delta_2 &\iff (s \models \delta_1) \wedge (s \models \delta_2), \\
s \models \delta_1 \text{ or } \delta_2 &\iff (s \models \delta_1) \vee (s \models \delta_2).
\end{aligned}$$

For example, `ci.status = done` holds in a state when the object `ci` has attribute `status` with value `done`. State propo-

TABLE I
MODELED COVERAGE OF RFC 8446 REQUIREMENTS BY GROUP

Group	Reqs.	Model	Cov.
Message order expectations	35	27	77.1%
Extension validity	45	36	80.0%
Negotiation consistency	74	56	75.7%
Authentication validity	47	32	68.1%
Cryptographic-context validity	21	14	66.7%
Session/resumption/post-handshake validity	67	50	74.6%
Others	38	14	36.8%
Total	327	229	70.0%

TABLE II
MODELED COVERAGE OF RFC 5246 REQUIREMENTS BY GROUP

Group	Reqs.	Model	Cov.
Message order expectations	10	8	80.0%
Extension validity	9	9	100.0%
Negotiation consistency	14	13	92.9%
Authentication validity	33	25	75.8%
Cryptographic-context validity	8	7	87.5%
Session/resumption/post-handshake validity	8	8	100.0%
Others	47	16	34.0%
Total	129	86	66.7%

sitions are evaluated at state positions in a trace, including the final state.

Action propositions are evaluated over the action tuple $a = (e, \ell, m, objs)$ from Section A. They inspect either the tuple metadata, such as `event`, `label`, and `ruleMsgType`, or the attributes of the object terms in `objs`. The satisfaction relation is:

$$\begin{aligned}
a \models \text{event} = e' &\iff e = e', \\
a \models \text{label} = \ell' &\iff \ell = \ell', \\
a \models \text{ruleMsgType} = m' &\iff m = m', \\
a \models o.x = v &\iff objs = \{\langle o : C \mid x : v, \dots \rangle, \dots\}, \\
a \models \text{not } \psi &\iff \neg(a \models \psi), \\
a \models \psi_1 \text{ and } \psi_2 &\iff (a \models \psi_1) \wedge (a \models \psi_2), \\
a \models \psi_1 \text{ or } \psi_2 &\iff (a \models \psi_1) \vee (a \models \psi_2).
\end{aligned}$$

For example, `event = error` selects rejection transitions, `label = removeKeyShareExtension` selects applications of that generated rewrite rule, and `ruleMsgType = server-hello` selects rule applications whose handled TLS message type is `server-hello`. A field predicate such as $o.x = v$ is evaluated over the matched object terms in `objs`, not over the successor state produced by the rewrite step.

Unlike state propositions, action propositions are evaluated only at transition positions. If a finite trace has k transitions, action propositions can be evaluated only at indices $0, \dots, k-1$; the final trace position is a state and has no outgoing action.

APPENDIX E SEMANTICS OF SCENARIO PROPERTY

We define scenario property semantics over the finite TLS traces introduced in Section A. For a trace τ with n transitions, positions range from 0 to n ; s_i denotes the state at position i , and a_i denotes the action from s_i to s_{i+1} . Scenario satisfaction is written $\tau, i \models \varphi$, meaning that scenario expression φ is satisfied from trace position i . State-only conditions are evaluated over s_i . Action-only conditions are evaluated over a_i and therefore require $i < n$. Mixed state/action conditions require both parts to hold at the same transition position.

For atomic expressions:

$$\begin{aligned}
\tau, i \models \text{true} &\iff 0 \leq i \leq n, \\
\tau, i \models \text{false} &\iff \text{false}, \\
\tau, i \models \delta &\iff 0 \leq i \leq n \wedge s_i \models \delta, \\
\tau, i \models \psi &\iff 0 \leq i < n \wedge a_i \models \psi, \\
\tau, i \models \delta \text{ and } \psi &\iff 0 \leq i < n \wedge s_i \models \delta \wedge a_i \models \psi, \\
\tau, i \models \text{any} &\iff 0 \leq i < n.
\end{aligned}$$

Here, δ ranges over state propositions and ψ ranges over action propositions, whose meanings are defined in Section D.

For composition:

$$\begin{aligned}
\tau, i \models \varphi_1; \varphi_2 &\iff i < n \wedge \tau, i \models \varphi_1 \wedge \tau, i+1 \models \varphi_2, \\
\tau, i \models \varphi^* &\iff \exists k. i \leq k \leq n \wedge \forall j. i \leq j < k \\
&\implies \tau, j \models \varphi, \\
\tau, i \models (\varphi_1 \mid \varphi_2) &\iff \tau, i \models \varphi_1 \vee \tau, i \models \varphi_2.
\end{aligned}$$

Because traces are finite, repetition in φ^* is always finite.

A trace satisfies a scenario property if the property holds at the initial trace position:

$$\tau \models \varphi \iff \tau, 0 \models \varphi.$$

This semantics treats scenario properties as existential finite-trace objectives: the analysis searches for some finite trace τ such that $\tau \models \varphi$.

APPENDIX F MANUAL FOLLOW-UP ANALYSIS OF MBEDTLS

The third case shows the manual path for target-specific state progression. The target is an MbedTLS 4.0.0 server configured for TLS 1.3 with client authentication, reconnection, and ticket resumption. The security-relevant outcome is a client-impersonation scenario: after a TLS 1.3 ticket-resumption flow, the server proceeds into a TLS 1.2 resumption-like state whose session secret is modeled as zero.

The initial RFC-conformance test is still generated automatically from the reusable library. The effective tester-side deviation removes a required `supported_versions`-related extension from a `ClientHello` in the resumption path:

```

BehaviorId : removeNecessaryExtension
Conditions : event = send and ruleMsgType = client-hello
Modification : remove(supported_versions)

```

RFC 8446 requires that, when a client sends a second `ClientHello` in response to a `HelloRetryRequest`, its `supported_versions` extension be identical to the one in the first `ClientHello`. Since this deviation removes `supported_versions` from the second `ClientHello`, a conforming server should reject it. In concrete execution, the MbedTLS server does not immediately reject it; instead, it sends subsequent handshake messages on a path that resembles TLS 1.2 resumption. The framework classifies the first execution as unexpected acceptance.

After this unexpected acceptance, the framework first applies the same violation-guided follow-up as in Section VII-D.

It maps the triggering deviation to the modeled mandatory-extension validation check, adds the corresponding noCheck behavior, and searches for a follow-up trace. However, this automated attempt does not reach the exploit condition. The observed execution therefore requires target-specific behavior beyond removing the RFC-labeled check used in the automated path.

The user then inspects the concrete execution and identifies the missing target-side behavior: after accepting the invalid resumption `ClientHello`, the `MbedTLS` proceeds as if it had entered a TLS 1.2 resumption mode with a zero-valued session secret. The user expresses this observed behavior as behavior deviation specifications. The first specification below captures the target-side state progression, using `setS` to encode the protocol-version, handshake-state, and session-state updates needed to model this complex target-side deviation.

```
BehaviorId : mbedTLWrongResumptionState
Conditions : event = receive and
             and ruleMsgType = client-hello
             and reconnectInProgress = true
Modification : setS(server.protocolVersion, TLS-12)
               and setS(server.handshakeState, V2S-READY)
               and setS(server.sessionState, zeroSecret)
```

The second deviation adjusts the controlled tester state so that the generated trace can continue consistently with the target’s wrong state progression. It does not modify the target; it describes how the tester should continue the conversation after the target has entered the abnormal resumption path.

```
BehaviorId : tryTLS12Resumption
Conditions : event = receive and
             and ruleMsgType = server-hello
             and reconnectInProgress = true
Modification : setS(client.protocolVersion, TLS-12)
               and setS(client.handshakeState, V2C-INIT)
               and setS(client.sessionState, zeroSecret)
```

The scenario property requires the initial RFC-violating input, the tester-side resumption adjustment, the target-side wrong-state behavior, and a completed resumption handshake.

```
ScenarioProperty [manualExploitAnalysis]:
any* ; (label = removeNecessaryExtension) ;
any* ; (label = mbedTLWrongResumptionState) ;
any* ; (label = tryTLS12Resumption) ;
any* ; (server.serverState = handshake-success
       and server.selectedVersion = TLS-12)
```

With these manually described deviations and the custom scenario property, the framework generates a new trace and executes the generated concrete test. The `MbedTLS` server completes the resumed handshake. In this state, the server carries forward the client-authenticated session identity with a zero-valued resumed session secret, thereby accepting the peer as the previously authenticated client. This is the client-impersonation condition expressed by the manual scenario.

This case demonstrates the role of manual behavior specification in complex exploit analysis. The initial RFC violation is detected by the same library-driven path used in the previous cases, but the exploit requires target-specific state information. Once the user expresses that behavior as behavior deviations and a scenario property, the framework uses the same trace-generation and concrete-execution to exploit target library.

APPENDIX G DETAILED RQ2 SPECIFICATION-VIOLATION AND VULNERABILITY RESULTS

Table III summarizes the numbers of RFC-violation test failures, completing handshakes, and distinct violated RFC requirements, and Table IV aggregates the resulting developer-confirmed bugs and assigned CVEs by library and TLS version. Table III expands these aggregate results at the RFC-requirement level. It shows which normative TLS requirements were violated by each recent stable library and whether each requirement-level finding became an observed RFC violation, a developer-confirmed bug, a CVE-assigned vulnerability.

Table III identifies each requirement by RFC number, section, and requirement text. To reduce repetition, the RFC column groups rows with the same RFC number while the Section column preserves the specific normative location for each requirement. The remaining columns are grouped by target library: `WolfSSL 5.8.2`, `GnuTLS 3.8.11`, and `MbedTLS 4.0.0`. Within each target group, the *RFC Violation* column records whether our tests observed a violation of the corresponding requirement, and the *CVE* column records whether that target-specific finding received a CVE. In the RFC Violation column, ● marks an observed violation that developers confirmed as a bug, ○ marks an observed violation without bug confirmation, and “—” marks no observed violation in our experiment. The CVE columns use O for presence and “—” for absence.

This requirement-level view shows why RFC-violation detection is useful beyond conformance reporting. A failed RFC requirement gives developers concrete evidence for triage: the same requirement-level violation can have different security consequences across implementations. In our results, some violations remain unconfirmed, some are confirmed by developers as bugs, and some are severe enough to receive CVEs. Therefore, the RFC requirement acts as a common unit of evidence that connects a concrete test failure to developer confirmation and, when applicable, vulnerability disclosure. This connection is important because it lets analysts prioritize specification violations that are more likely to expose security-relevant implementation behavior.

For example, the TLS 1.3 duplicate-extension requirement in RFC 8446 Section 4.2 is violated by all three libraries in our experiment, but only the `WolfSSL` instance is CVE-assigned in this requirement-level view. Other rows show the same escalation path for different requirements, including PSK binder validation in `GnuTLS` and `supported_versions` handling in `MbedTLS`. Thus, the table demonstrates that requirement-level violations are not merely formatting or interoperability issues; they identify specification checks whose absence can become security-relevant in concrete implementations.

Table IV gives the per-bug metadata behind the confirmed-bug and CVE counts in Table IV. Each row corresponds to one newly discovered, developer-confirmed bug. The target column identifies the affected library and version, the behavior-deviation column records the P1–P4 pattern that triggered the failing test, and the bug-reference column links the row to the

corresponding developer report or advisory. When a CVE was assigned, the CVE, CVSS, and vulnerability-category columns report the public identifier, severity, and security consequence; otherwise, these fields use “-”.

The per-bug rows identify the 16 newly discovered, developer-confirmed bugs found by our framework, including eight CVE-assigned vulnerabilities across WolfSSL, MbedTLS, and GnuTLS. These rows support two observations. First, the confirmed bugs are not concentrated in a single deviation pattern: all four parameterized behavior-deviation patterns used in Section VIII-A contribute to at least one confirmed bug. Second, the CVE-assigned rows show that the framework exposes more than memory-safety failures, spanning client impersonation, key compromise, downgrade, denial of service, and loss of perfect forward secrecy. Together, these results indicate that behavior-deviation testing finds security-relevant bugs across both deviation patterns and vulnerability categories, rather than only crash-inducing inputs.

APPENDIX H DETAILED RQ3 EXISTING-TOOL COMPARISON

Table V expands the RQ3 comparison in Table V from tool-level overlap counts to library/version-level evidence. The main table compares the sets of RFC violations found by TLS-Anvil, TLS-Attacker, and DY-Fuzzer with those found by our framework over matched target settings. This appendix table shows, for each such setting, whether the baseline-reported bug(s) were reproduced and how many additional RFC-violation failures our generated tests exposed. The comparison is scoped to behaviors represented by our RFC-level TLS 1.2/1.3 handshake model and should not be read as a global ranking of the tools.

Each row corresponds to one target library/version setting taken from TLS-Anvil, TLS-Attacker, or DY-Fuzzer. The *Target* column names the implementation and version; *Reproduced* records the status of baseline-reported bug reproduction; *Addl. bugs* counts additional RFC violations found by our framework within that row; and *Test Fails* reports the number of generated RFC-violation tests that failed on that target. In the *Reproduced* column, $Y(n)$ means all n reported bugs for the setting were reproduced, $N(n)$ means none of the n reported bugs were reproduced, $P(a/b)$ means a of b reported bugs were reproduced, and “-” marks settings for which the baseline comparison did not report a prior bug. *N/A* marks counts unavailable for a setting outside the currently supported comparison harness.

The row-level results clarify why the aggregate overlap in Table V contains both reproduced and non-reproduced baseline findings. Reading the table in order, the TLS-Anvil block contains reproduced, partially reproduced, unreproduced, and no-prior-bug settings; the TLS-Attacker block contains reproduced and unreproduced settings, including one unsupported MatrixSSL setting; and the DY-Fuzzer block contains reproduced, unreproduced, and partially reproduced settings over OpenSSL and WolfSSL versions. Rows marked Y or

P show cases where deviation-guided RFC testing reproduces baseline-reported bugs whose triggers fit the modeled handshake behavior, while rows marked N expose scope boundaries such as record-layer behavior, cryptographic value construction, legacy protocol features, or unsupported builds. At the same time, every evaluated row with a numeric *Addl. bugs* value contains at least one additional RFC violation, including settings where the baseline comparison reported no prior bug. This pattern supports the RQ3 claim that modeling RFC requirements in handshake context and instantiating parameterized behavior deviations can expose violations beyond prior tool-specific tests, while the non-reproduced rows make the model boundary explicit.

APPENDIX I RFC REQUIREMENT LABELS

This appendix lists the RFC requirements [23], [24] encoded as label-indexed predicates used by the receive rules in Section IV. Each table row is assigned a globally unique numeric label identifier. The label is a stable handle for the modeled predicate; the RFC reference, message type, and description provide the protocol context in which that predicate is checked.

Receive rules call `checkRFC` with a requirement label, the received message, and the receiver object matched by the rule. The label selects the predicate to evaluate, while the receiver object supplies the endpoint context needed by state-dependent requirements, such as supported parameters or already negotiated values. For a message field $f_1(m_1)$, a representative predicate has the following form:

$$\begin{aligned} & \text{eq checkRFC}(l_{f_1}, f_1(m_1) \cdots f_n(m_n), \\ & \quad \langle o_{dst} : \text{NodeType} \mid a_1 : v_1, \dots, a_m : v_m \rangle) \\ & = \text{checkMessage}(m_1, \text{opr}(v_1, \dots, v_m)). \end{aligned}$$

TABLE III
REQUIREMENT-LEVEL RQ2 OUTCOMES FOR RECENT STABLE TLS LIBRARIES

RFC	Section	RFC Requirement	WolfSSL (5.8.2)		GnuTLS (3.8.11)		MbedTLS (4.0.0)	
			RFC Violation	CVE	RFC Violation	CVE	RFC Violation	CVE
5246	7.4.3	The hash and signature algorithms used in the signature MUST be one of those present in the supported_signature_algorithms field of the CertificateRequest message.	●	O	–	–	●	O
	7.4.1.4	An extension type MUST NOT appear in the ServerHello unless the same extension type appeared in the corresponding ClientHello.	●	–	○	–	○	–
	7.4.1.4	There MUST NOT be more than one extension of the same type.	●	–	–	–	●	–
	7.4.2	The certificate MUST be appropriate for the negotiated cipher suite's key exchange algorithm and any negotiated extensions.	●	–	–	–	–	–
	7.4.2	The server MUST send a Certificate message whenever the agreed-upon key exchange method uses certificates for authentication (this includes all key exchange methods defined in this document except DH_anon).	●	–	–	–	–	–
	7.4.3	This message will be sent immediately after the server Certificate message (or the ServerHello message, if this is an anonymous negotiation).	●	–	–	–	–	–
	7.4.7	This message is always sent by the client. It MUST immediately follow the client certificate message, if it is sent.	●	–	–	–	–	–
	7.4.4	A non-anonymous server can optionally request a certificate from the client, if appropriate for the selected cipher suite. This message, if sent, will immediately follow the ServerKeyExchange message (if it is sent; otherwise, this message follows the server's Certificate message).	●	–	–	–	–	–
8446	4.1.4	The server's extensions MUST contain supported_versions. Additionally, it SHOULD contain the minimal set of extensions necessary for the client to generate a correct ClientHello pair.	●	–	–	–	–	–
	4.1.2	The client will also send a ClientHello when the server has responded to its ClientHello with a HelloRetryRequest. In that case, the client MUST send the same ClientHello without modification.	○	–	●	–	●	–
	4.2.8	Each KeyShareEntry value MUST correspond to a group offered in the supported_groups extension and MUST appear in the same order.	●	–	–	–	●	–
	4.2.8	Clients MUST NOT offer any KeyShareEntry values for groups not listed in the client's supported_groups extension.	●	–	–	–	–	–
	4.2.8	Otherwise, when sending the new ClientHello, the client MUST replace the original key_share extension with one containing only a new KeyShareEntry for the group indicated in the selected_group field of the triggering HelloRetryRequest.	○	–	○	–	○	–
	4.2.8	If using (EC)DHE key establishment, servers offer exactly one KeyShareEntry in the ServerHello. This value MUST be in the same group as the KeyShareEntry value offered by the client that the server has selected for the negotiated key exchange.	●	O	–	–	–	–
	4.2.8	Clients MUST NOT offer multiple KeyShareEntry values for the same group.	●	O	–	–	–	–
	4.2.9	psk_dhe_ke: PSK with (EC)DHE key establishment. In this mode, the client and server MUST supply key_share values as described in Section 4.2.8.	●	O	–	–	–	–
	4.4.3	If the CertificateVerify message is sent by a server, the signature algorithm MUST be one offered in the client's signature_algorithms extension unless no valid certificate chain can be produced without unsupported algorithms.	●	O	–	–	–	–
	4.2	There MUST NOT be more than one extension of the same type in a given extension block.	●	O	●	–	●	–
	4.2.3	In particular, MD5, SHA-224, and DSA MUST NOT be used.	–	–	–	–	○	–
	4.1.2	legacy_version field MUST be set to 0x0303.	–	–	●	–	–	–
	4.1.2	For every TLS 1.3 ClientHello, this vector MUST contain exactly one byte, set to zero, which corresponds to the "null" compression method in prior versions of TLS.	–	–	○	–	–	–
	4.2.1	The extension contains a list of supported versions in preference order, with the most preferred version first. Implementations of this specification MUST send this extension in the ClientHello.	–	–	–	–	●	O
	4.2.11	Prior to accepting PSK key establishment, the server MUST validate the corresponding binder value.	–	–	●	O	–	–

● marks RFC violations confirmed by developers as bugs; ○ marks observed RFC violations without bug confirmation. In the CVE columns, O marks presence; "–" marks absence in the corresponding column.

TABLE IV
RQ2 CVE METADATA FOR NEWLY DISCOVERED BUGS

Target	Behavior Deviation	Bug Reference	CVE	CVSS	Vulnerability Category
MbedTLS(4.0.0)	P2	[1]	CVE-2026-34873 [2]	Critical (9.1)	Client impersonation
WolfSSL(5.8.4)	P2	[3]	CVE-2026-3230 [4]	Low (2.7)	Key compromise
GnuTLS(3.8.11)	P3	[5]	CVE-2026-1584 [6]	High (7.5)	Memory crash
WolfSSL(5.8.2)	P3	[7]	CVE-2025-12889 [8]	Medium (5.4)	Downgrade attack
WolfSSL(5.8.2)	P3	[9]	CVE-2025-11936 [10]	Medium (5.3)	Denial of service
WolfSSL(5.8.2)	P2	[11]	CVE-2025-11935 [12]	High (7.5)	Non-perfect forward secrecy
WolfSSL(5.8.2)	P3	[13]	CVE-2025-11934 [14]	Low (2.7)	Downgrade attack
WolfSSL(5.8.2)	P1	[15]	CVE-2025-11933 [16]	Medium (6.5)	Denial of service
WolfSSL(5.8.2)	P3	[17]	–	–	–
WolfSSL(5.8.4)	P4	[18]	–	–	–
WolfSSL(5.8.4)	P4	[18]	–	–	–
WolfSSL(5.8.4)	P4	[18]	–	–	–
WolfSSL(5.8.4)	P3	[19]	–	–	–
WolfSSL(5.8.4)	P1	[20]	–	–	–
WolfSSL(5.8.4)	P1	[21]	–	–	–
MbedTLS(4.0.0)	P1	[22]	–	–	–

CVSS reports public CVSS v3.1 base severity and score in the cited CVE records. Bugs without a linked CVE use "–".

TABLE V
REPRODUCTION AND ADDITIONAL BUG DETECTION UNDER
EXISTING-TOOL LIBRARY/VERSION SETTINGS

Tool	Target	Repro- duced	Addl. bugs	Test Fails
TLS-Anvil	BearSSL 0.6	N(1)	1	8
	BoringSSL 3945	–	13	31
	Botan 2.17.3	–	2	32
	GnuTLS 3.7.0	Y(1)	6	11
	LibreSSL 3.2.3	P(1/2)	1	15
	MatrixSSL 4.3.0	P(7/11)	13	159
	MbedTLS 2.25.0	N(1)	2	17
	NSS 3.6	–	4	35
	OpenSSL 1.1.1i	–	6	23
	Rustls 0.19.0	Y(1)	2	17
	s2n 0.10.24	Y(1)	6	12
	tlslite-ng 0.8.0-a39	–	2	4
	WolfSSL 4.5.0	Y(5)	13	136
TLS- Attacker	Botan 1.11.21	N(1)	1	26
	OpenSSL 1.1.0-pre1	Y(1)	5	38
	MatrixSSL 3.8.1	N(2)	N/A	N/A
	GnuTLS 3.4.9	Y(1)	3	52
	Botan 1.11.28	Y(1)	1	26
	Botan 1.11.30	N(1)	5	63
DY-Fuzzer	OpenSSL 1.1.1j	Y(1)	6	23
	WolfSSL 5.1.0	Y(1)	16	163
	WolfSSL 5.3.0	N(1)	11	92
	WolfSSL 5.4.0	N(1)	9	100
	WolfSSL 5.5.0	P(1/2)	13	121

Reproduced: Y(n)=all n reported bugs reproduced, N(n)=none reproduced,
P(a/b)= a of b reproduced, –=no prior reported bug. Addl. bugs count unique RFC
violations within each library/version setting; N/A marks unavailable counts.

TABLE VI: RFC requirement labels encoded by the Maude TLS model.

Message Type	Label ID	RFC	Section	Description
ClientHello	label11	8446	4.2.8	KeyShareEntry values MUST correspond to groups offered in the "supported_groups" extension and MUST appear in the same order.
ClientHello	label12	8446	4.1.2	TLS 1.3 forbids renegotiation; reject TLS 1.2 renegotiation signaling on TLS 1.3 ClientHello paths.
ClientHello	label13	8446	4.2	There MUST NOT be more than one extension of the same type in a given extension block.
ClientHello	label14	8446	4.1.1	If no supported client/server parameters overlap, the server MUST abort with handshake_failure or insufficient_security.
ClientHello	label15	8446	4.1.1 and 4.2.7	The server must be able to negotiate supported parameters, including a mutually supported named group when groups are offered.
ClientHello	label16	8446	4.1.2	The TLS 1.3 ClientHello compression vector MUST contain exactly one zero byte, the null compression method.
ClientHello	label17	8446	9.2	A TLS 1.3 ClientHello with supported_versions 0x0304 MUST meet mandatory extension requirements.
ClientHello	label18	8446	4.2.3	Deprecated signature algorithms MUST NOT be offered or negotiated; MD5, SHA-224, and DSA MUST NOT be used.
ClientHello	label19	8446	4.1.2 and 4.2.1	TLS 1.3 ClientHello uses legacy_version 0x0303 and identifies TLS 1.3 via supported_versions 0x0304.
ClientHello	label110	8446	4.2.8	Clients MUST NOT offer multiple KeyShareEntry values for the same group.
ClientHello	label111	8446	4.2.3	If the server authenticates via certificate and the client omitted signature_algorithms, the server MUST abort.
ClientHello	label112	8446	B.4	TLS 1.3 cipher suites cannot be used for TLS 1.2, and TLS 1.2 or lower cipher suites cannot be used with TLS 1.3.
ClientHello	label113	8446	4.2.9	A client offering pre_shared_key MUST also provide psk_key_exchange_modes.
ClientHello	label114	8446	4.2.10	A client using early data MUST supply both pre_shared_key and early_data extensions.
ClientHello	label115	8446	4.2.9	In psk_dhe_ke, client and server MUST supply key_share values; in psk_ke, the server MUST NOT use key_share.
ClientHello	label116	8446	4.1.2 and 6	ClientHello vector bounds must parse correctly; syntax length failures terminate with decode_error.
ClientHello	label117	8446	4	The expected handshake message is ClientHello; unexpected handshake order or type aborts with unexpected_message.
ClientHello	label118	8446	5	ClientHello is processed as a handshake record; unexpected record content types terminate with unexpected_message.
HelloRetryRequest	label119	8446	4.1.4	HelloRetryRequest MUST only contain extensions valid for HelloRetryRequest.
HelloRetryRequest	label120	8446	4.1.4	HelloRetryRequest MUST contain supported_versions.
HelloRetryRequest	label121	8446	4.1.4	The selected group must cause a changed ClientHello.
HelloRetryRequest	label122	8446	4.1.4	The selected group must be from the original supported_groups.
HelloRetryRequest	label123	8446	4.1.4	A client MUST abort on a second HelloRetryRequest in the same connection.
HelloRetryRequest	label124	8446	4.1.4	HelloRetryRequest legacy_version MUST be TLS 1.2.
HelloRetryRequest	label125	8446	4.1.4	HelloRetryRequest cipher suite MUST be one offered by the client.
HelloRetryRequest	label126	8446	4.1.4	HelloRetryRequest compression method MUST be null.
HelloRetryRequest	label127	8446	4.1.4	HelloRetryRequest legacy_session_id_echo MUST match ClientHello.
HelloRetryRequest	label128	8446	4.2	There MUST NOT be duplicate extensions.
HelloRetryRequest	label129	8446	4.2.1	HelloRetryRequest selected_version MUST have been offered by the client.
HelloRetryRequest	label130	8446	4 and 6	HelloRetryRequest syntax lengths must parse correctly.
HelloRetryRequest	label131	8446	4	The expected handshake message is HelloRetryRequest.
HelloRetryRequest	label132	8446	5	HelloRetryRequest is carried in a handshake record.
ServerHello	label133	8446	4.1.3 and 4.2.1	ServerHello legacy_version and supported_versions must indicate TLS 1.3 correctly.
ServerHello	label134	8446	4.2.1	ServerHello selected_version must be one offered by the client.
ServerHello	label135	8446	4.2	There MUST NOT be duplicate extensions.
ServerHello	label136	8446	4.1.3	ServerHello cipher suite MUST be one offered by the client.
ServerHello	label137	8446	4.2	ServerHello MUST NOT contain extensions forbidden for this message.
ServerHello	label138	8446	4.2	ServerHello extension responses require corresponding client requests.
ServerHello	label139	8446	4.2.8	ServerHello key_share MUST match a client offered group.
ServerHello	label140	8446	4.2.8	ServerHello key_share group MUST appear in client supported_groups.
ServerHello	label141	8446	4.2.9	ServerHello MUST NOT send key_share when psk_ke is used.
ServerHello	label142	8446	4.1.3	legacy_session_id_echo MUST match the ClientHello session id.
ServerHello	label143	8446	4.1.3	legacy_compression_method MUST be null for TLS 1.3 ServerHello.
ServerHello	label144	8446	4.2.9	The server MUST NOT send psk_key_exchange_modes.
ServerHello	label145	8446	4.2.1	ServerHello MUST contain supported_versions.
ServerHello	label146	8446	4.2.9	psk_dhe_ke requires a server key_share.
ServerHello	label147	8446	4.2.11	selected_identity must be in the offered PSK range.
ServerHello	label148	8446	4.2.8	ServerHello must contain key_share unless PSK-only mode applies.
ServerHello	label149	8446	4.2.11	If PSK was offered, selected PSK extension must be present.
ServerHello	label150	8446	4 and 6	ServerHello syntax lengths must parse correctly.
ServerHello	label151	8446	4	The expected handshake message is ServerHello.
EncryptedExtensions	label152	8446	4.3.1	EncryptedExtensions MUST NOT contain forbidden extensions.
EncryptedExtensions	label153	8446	4 and 6	EncryptedExtensions syntax lengths must parse correctly.
EncryptedExtensions	label154	8446	4	The expected handshake message is EncryptedExtensions.
EncryptedExtensions	label155	8446	5	EncryptedExtensions is read under application-data record protection.
CertificateRequest	label156	8446	4.3.2	CertificateRequest MUST include signature_algorithms and only allowed extensions.
CertificateRequest	label157	8446	4.2	There MUST NOT be duplicate CertificateRequest extensions.
CertificateRequest	label158	8446	4 and 6	CertificateRequest syntax lengths must parse correctly.
CertificateRequest	label159	8446	4	CertificateRequest or Certificate may be expected in this state.

Message Type	Label ID	RFC	Section	Description
CertificateRequest	label160	8446	5	CertificateRequest is read under application-data record protection.
Certificate	label161	8446	4.4.2	Server certificate_list MUST be non-empty.
Certificate	label162	8446	4.4.2.2	Certificates MUST NOT use MD5 or SHA-1 signatures.
Certificate	label163	8446	4.4.2.2	Certificate public key/signature algorithms must be acceptable.
Certificate	label164	8446	4.4.2.2	Certificate signatures must verify.
Certificate	label165	8446	4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label166	8446	4	Certificate handshake type must be valid for the current state.
Certificate	label167	8446	5	Certificate is read under application-data record protection.
Certificate	label168	8446	4.4.2.2	Client certificate signatures MUST NOT use MD5 or SHA-1.
Certificate	label169	8446	4.4.2.2	Client certificates must use acceptable signature algorithms.
Certificate	label170	8446	4.4.2.2	Client certificate signatures must verify.
Certificate	label171	8446	4.4.2	Client CertificateRequest context must match.
Certificate	label172	8446	4.4.2	Client certificate_list must satisfy local client-certificate policy.
Certificate	label173	8446	4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label174	8446	4	The expected handshake message is Certificate.
Certificate	label175	8446	5	Client Certificate is read under application-data record protection.
CertificateVerify	label176	8446	4.4.3	CertificateVerify signature algorithm MUST NOT use SHA-1.
CertificateVerify	label177	8446	4.4.3	CertificateVerify signature algorithm must be offered.
CertificateVerify	label178	8446	4.4.3	CertificateVerify signature must verify against the transcript.
CertificateVerify	label179	8446	4 and 6	CertificateVerify syntax lengths must parse correctly.
CertificateVerify	label180	8446	4	The expected handshake message is CertificateVerify.
CertificateVerify	label181	8446	5	CertificateVerify is read under application-data record protection.
Finished	label182	8446	4.4.4	Finished verify_data must be correct.
Finished	label183	8446	4 and 6	Finished syntax lengths must parse correctly.
Finished	label184	8446	4	The expected handshake message is Finished.
Finished	label185	8446	5	Finished is read under application-data record protection.
CertificateRequest	label186	8446	4.6.2	Post-handshake CertificateRequest requires prior post_handshake_auth support.
KeyUpdate	label187	8446	4.6.3	KeyUpdate request_update value must be known.
KeyUpdate	label188	8446	4 and 6	KeyUpdate syntax lengths must parse correctly.
KeyUpdate	label189	8446	4	The expected handshake message is KeyUpdate.
ClientHello	label190	5246	7.4.1.2	ClientHello.client_version MUST be a supported TLS version.
ClientHello	label191	5246	7.4.1.2	compression_methods MUST contain CompressionMethod.null.
ClientHello	label192	5246	7.4.1.2	The server MUST select an acceptable cipher suite.
ClientHello	label193	5246	7.4.1.2	The server MUST select an acceptable compression method.
ClientHello	label194	5246	7.4.1.4	There MUST NOT be more than one extension of the same type.
ClientHello	label195	5246	7.4.1.4.1	The anonymous signature value MUST NOT appear in signature_algorithms.
ClientHello	label196	5246	7.4.4	Certificate authentication requires signature_algorithms in this model.
ClientHello	label197	5246	7.4.1.4	ClientHello extensions must be extensions supported by this TLS 1.2 model.
ClientHello	label198	5246	7.4.1.4.1	Signature algorithms must overlap with server-supported algorithms.
ClientHello	label199	5746	3.4	Client renegotiation_info MUST match previous verify_data.
ClientHello	label1100	5246	7.4 and 6	ClientHello syntax lengths must parse correctly.
ClientHello	label1101	5246	7.4	The expected handshake message is ClientHello.
ClientHello	label1102	5246	6.2.1	ClientHello is carried in a handshake record.
ServerHello	label1103	5246	7.4.1.3	ServerHello.server_version MUST be TLS 1.2 in this model.
ServerHello	label1104	5246	7.4.1.3	The cipher suite selected by the server MUST be offered by the client.
ServerHello	label1105	5246	7.4.1.3	The compression method selected by the server MUST be offered by the client.
ServerHello	label1106	5246	7.4.1.4	ServerHello MUST NOT contain unsolicited extensions, except renegotiation_info.
ServerHello	label1107	5246	7.4.1.4.1	Servers MUST NOT send supported_signature_algorithms.
ServerHello	label1108	4492	5.1	Servers MUST NOT negotiate supported_groups in ServerHello in this model.
ServerHello	label1109	5246	7.4.1.4	There MUST NOT be duplicate ServerHello extensions.
ServerHello	label1110	5746	3.6	Server renegotiation_info MUST match previous verify_data.
ServerHello	label1111	5246	7.4 and 6	ServerHello syntax lengths must parse correctly.
ServerHello	label1112	5246	7.4	The expected handshake message is ServerHello.
ServerHello	label1113	5246	6.2.1	ServerHello is carried in a handshake record.
Certificate	label1114	5246	7.4.2	Server certificates MUST be appropriate for the negotiated cipher suite.
Certificate	label1115	5246	7.4.2	Server certificate chains must be issued by an accepted authority in this model.
Certificate	label1116	5246	7.4.2	Server certificate signatures must verify.
Certificate	label1117	5246	7.4.2	If signature_algorithms was sent, certificate signatures must use an offered pair.
Certificate	label1118	5246	7.4.2	A non-anonymous server MUST send a non-empty certificate_list.
Certificate	label1119	5246	7.4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label1120	5246	7.4	The expected handshake message is Certificate.
Certificate	label1121	5246	6.2.1	Certificate is carried in a handshake record.
Certificate	label1122	5246	7.4.6	Client certificates MUST be appropriate for the negotiated cipher suite.
Certificate	label1123	5246	7.4.6	Client certificates should be issued by one of the requested authorities.
Certificate	label1124	5246	7.4.6	Client certificate signatures must verify.
Certificate	label1125	5246	7.4.6	Client certificates must use a requested signature algorithm.
Certificate	label1126	5246	7.4.6	This model requires a non-empty client certificate_list when client auth is requested.
Certificate	label1127	5246	7.4 and 6	Certificate syntax lengths must parse correctly.
Certificate	label1128	5246	7.4	The expected handshake message is Certificate.
Certificate	label1129	5246	6.2.1	Certificate is carried in a handshake record.
ClientKeyExchange	label1130	5246	7.4.7	ClientKeyExchange syntax lengths must parse correctly.
ClientKeyExchange	label1131	5246	7.4	The expected handshake message is ClientKeyExchange.
ClientKeyExchange	label1132	5246	6.2.1	ClientKeyExchange is carried in a handshake record.

Message Type	Label ID	RFC	Section	Description
CertificateVerify	label1133	5246	7.4.8	CertificateVerify signatures must verify over the handshake transcript.
CertificateVerify	label1134	5246	7.4.8	CertificateVerify signature algorithm must be requested.
CertificateVerify	label1135	5246	7.4 and 6	CertificateVerify syntax lengths must parse correctly.
CertificateVerify	label1136	5246	7.4	The expected handshake message is CertificateVerify.
CertificateVerify	label1137	5246	6.2.1	CertificateVerify is carried in a handshake record.
ServerKeyExchange	label1138	5246	7.4.3	ServerKeyExchange signatures must verify when present.
ServerKeyExchange	label1139	5246	7.4.3	Ephemeral ECDH parameters must use an accepted group.
ServerKeyExchange	label1140	5246	7.4 and 6	ServerKeyExchange syntax lengths must parse correctly.
ServerKeyExchange	label1141	5246	7.4	The expected handshake message is ServerKeyExchange.
ServerKeyExchange	label1142	5246	6.2.1	ServerKeyExchange is carried in a handshake record.
CertificateRequest	label1143	5246	7.4 and 6	CertificateRequest syntax lengths must parse correctly.
CertificateRequest	label1144	5246	6.2.1	CertificateRequest is carried in a handshake record.
CertificateRequest	label1145	5246	7.4	CertificateRequest or ServerHelloDone may be expected at this point.
ServerHelloDone	label1146	5246	7.4 and 6	ServerHelloDone syntax lengths must parse correctly.
ServerHelloDone	label1147	5246	6.2.1	ServerHelloDone is carried in a handshake record.
ServerHelloDone	label1148	5246	7.4	The expected handshake message is ServerHelloDone.
ChangeCipherSpec	label1149	5246	7.1	ChangeCipherSpec syntax lengths must parse correctly.
ChangeCipherSpec	label1150	5246	6.2.1	ChangeCipherSpec is carried in a change_cipher_spec record.
Finished	label1151	5246	7.4.9	Finished verify_data must match the handshake transcript and master secret.
Finished	label1152	5246	7.4 and 6	Finished syntax lengths must parse correctly.
Finished	label1153	5246	7.4	The expected handshake message is Finished.
Finished	label1154	5246	6.2.1	Finished is carried in a handshake record.

REFERENCES

- [1] Mbed TLS, “Mbed TLS security advisory 2026-03.” <https://mbed-tls.readthedocs.io/en/latest/security-advisories/>, 2026. Client impersonation while resuming a TLS 1.3 session.
- [2] National Vulnerability Database, “CVE-2026-34873.” <https://nvd.nist.gov/vuln/detail/CVE-2026-34873>, 2026.
- [3] wolfSSL, “wolfSSL pull request #9754.” <https://github.com/wolfSSL/wolfssl/pull/9754>, 2026.
- [4] National Vulnerability Database, “CVE-2026-3230.” <https://nvd.nist.gov/vuln/detail/CVE-2026-3230>, 2026.
- [5] GnuTLS, “GnuTLS issue #1790.” <https://gitlab.com/gnutls/gnutls/-/issues/1790>, 2026.
- [6] National Vulnerability Database, “CVE-2026-1584.” <https://nvd.nist.gov/vuln/detail/CVE-2026-1584>, 2026.
- [7] wolfSSL, “wolfSSL pull request #9395.” <https://github.com/wolfSSL/wolfssl/pull/9395>, 2026.
- [8] National Vulnerability Database, “CVE-2025-12889.” <https://nvd.nist.gov/vuln/detail/CVE-2025-12889>, 2025.
- [9] wolfSSL, “wolfSSL pull request #9117.” <https://github.com/wolfSSL/wolfssl/pull/9117>, 2026.
- [10] National Vulnerability Database, “CVE-2025-11936.” <https://nvd.nist.gov/vuln/detail/CVE-2025-11936>, 2025.
- [11] wolfSSL, “wolfSSL pull request #9112.” <https://github.com/wolfSSL/wolfssl/pull/9112>, 2026.
- [12] National Vulnerability Database, “CVE-2025-11935.” <https://nvd.nist.gov/vuln/detail/CVE-2025-11935>, 2025.
- [13] wolfSSL, “wolfSSL pull request #9113.” <https://github.com/wolfSSL/wolfssl/pull/9113>, 2026.
- [14] National Vulnerability Database, “CVE-2025-11934.” <https://nvd.nist.gov/vuln/detail/CVE-2025-11934>, 2025.
- [15] wolfSSL, “wolfSSL pull request #9132.” <https://github.com/wolfSSL/wolfssl/pull/9132>, 2026.
- [16] National Vulnerability Database, “CVE-2025-11933.” <https://nvd.nist.gov/vuln/detail/CVE-2025-11933>, 2025.
- [17] wolfSSL, “wolfSSL issue #9378.” <https://github.com/wolfSSL/wolfssl/issues/9378>, 2026.
- [18] wolfSSL, “wolfSSL issue #9650.” <https://github.com/wolfSSL/wolfssl/issues/9650>, 2026.
- [19] wolfSSL, “wolfSSL issue #9651.” <https://github.com/wolfSSL/wolfssl/issues/9651>, 2026.
- [20] wolfSSL, “wolfSSL issue #9652.” <https://github.com/wolfSSL/wolfssl/issues/9652>, 2026.
- [21] wolfSSL, “wolfSSL issue #9655.” <https://github.com/wolfSSL/wolfssl/issues/9655>, 2026.
- [22] Mbed TLS, “Mbed TLS issue #6804.” <https://github.com/Mbed-TLS/mbedtls/issues/6804>, 2026.
- [23] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2.” RFC 5246, 2008.
- [24] E. Rescorla, “The transport layer security (tls) protocol version 1.3.” RFC 8446, 2018.